Chapter 4
Query Processing and Operator Algorithms

How scans, sorts, joins and aggregates actually run, and what each one costs

Advanced Data Management

Based on the Chapter 4 lecture, class examples and Lab 4

How to read this chapter

Chapter 1 showed that a query becomes a tree of operators and that the executor runs that tree with `open/next/close`. Chapters 2 and 3 gave us the cost currency (pages) and the access paths (scan or index). This chapter opens each operator and asks two questions: *how does it actually run*, and *how many pages does it touch*? The two operators that matter most are sorting and joining, because they are where a query's cost concentrates. By the end you should be able to count the I/O of an external sort and of each of the three join algorithms, say which join wins in a given situation, and read a PostgreSQL plan operator by operator. All the plans shown are from a real run on the lab database (about 20,000 customers and 500,000 orders).

1 Refresher: Sorting

1.1 Why sorting matters here

Sorting looks like a side topic. It is not: several operators are built on it.

- `ORDER BY` obviously, and `DISTINCT` (sort, then drop adjacent duplicates).
- Sort-merge join, coming up in Section 5.
- Sort-based grouping and aggregation, Section 6.

So we first recall ordinary merge sort, then adapt it to data that does not fit in memory. The adaptation is the first real operator we study.

1.2 Merge sort

Divide the input into halves, sort each half recursively, then **merge**. The merge step is the part we will reuse, so look at it closely: given two already-sorted lists, walk both with one pointer each and emit the smaller front ele-

ment, advancing that pointer. Every element is looked at once, so merging is *linear* in the total size, and it is *streaming*: it never needs both lists fully in memory, just their current fronts.



Total cost is $O(n \log n)$: $\log_2 n$ levels of halving, and $O(n)$ merge work per level.

Notice that nothing in the merge step requires exactly two inputs. With k sorted lists, keep k pointers, and at each step emit the smallest front among the k. That generalisation is what makes the database version work.

Key insight. Merging k sorted lists is cheap and streaming. A database turns "sort a huge file" into "make sorted pieces, then merge them".

1.3 The database twist

The data to sort is often larger than memory. We have a fixed budget of M buffer pages to work in, and the file has B pages with $B \gg M$. We cannot load it all, so we sort in pieces that fit and merge the pieces from disk. This is **external merge sort**, and it is the same idea as merge sort with one constraint added: every pass over the data costs $2B$ page I/Os (read everything, write everything), so the game is to minimise the number of passes.

2 The Executor and the Iterator Model

2.1 The iterator model, recalled

Mechanism: Iterator (Volcano) model

Every operator implements three calls:

- `open()`: prepare (allocate memory, open children).
- `next()`: return the next output tuple, calling `next()` on children as needed; return nothing when exhausted.
- `close()`: release resources.

The top operator is pulled for tuples; requests flow down the tree, tuples flow up. Most operators are **pipelined**: a tuple streams through without being stored.

Pipelining is why a query on a billion rows can start returning results immediately and why intermediate results usually never touch disk. A selection reads one tuple from its child, tests it, and either passes it up or asks for another. Nothing accumulates.

2.2 Pipeline breakers

Some operators cannot work that way. They must see *all* of their input before producing *any* output:

- **Sort.** It cannot emit the smallest element until it has seen the last one, because the last one might be the smallest.
- **Hash build.** A hash join must build the complete hash table on one input before it can probe with the other.
- **Grouping and aggregation over unsorted input.** A `SUM` per group is not final until the last row of that group has arrived, and with unsorted input that could be the last row of the file.

Concept: Why name them?

Pipeline breakers **materialise** data: they hold their whole input in memory, or spill it to disk when it does not fit. They are where the I/O cost of a plan concentrates. When you read a plan, find the breakers first; the rest is usually cheap streaming around them.

An analogy: a pipelined operator is a worker on an assembly line who handles each item as it passes. A pipeline breaker is a worker who has to collect every item into a pile, do something to the pile, and only then start passing items on. The pile needs somewhere to sit, and if it is bigger than the workbench, it goes on the floor (disk).

Summary

Operators are iterators; tuples stream up the tree one at a time. Sort, hash build, and unsorted aggregation are pipeline breakers: they consume everything before emitting anything, so they materialise and may spill to disk. That is where a plan's cost concentrates.

3 Simple Operators

Operator	How it runs	Cost
Sequential scan	read every page of the table	B
Index scan	descend the index, fetch matching rows (Chapter 3)	$h+$ matches
Selection σ	test each incoming tuple against the predicate; pass or drop	0 extra
Projection π	keep the chosen columns of each tuple; drop the rest	0 extra
Duplicate elimination	sort, then drop adjacent equals; or hash and drop collisions	a sort or a hash

Selection and projection are pipelined and cost nothing in I/O beyond what their child already paid; they are pure per-tuple CPU work. Duplicate elimination is different: to know whether a tuple is a duplicate you must have seen

all the others, so it needs either sorting (equal values become adjacent) or a hash table (equal values collide). That is the first appearance of a pattern that runs through the whole chapter: *to find equal things, either sort so they sit together, or hash so they land together.*

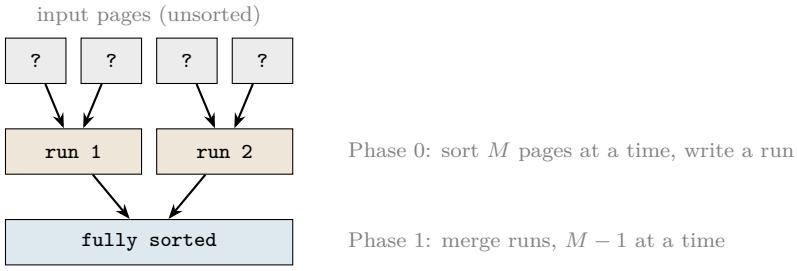
4 External Merge Sort

4.1 The plan

Mechanism: External merge sort with M buffer pages

- **Phase 0, run generation.** Read M pages into the buffer, sort them in memory (any in-memory sort), write them out as one **sorted run**. Repeat for the next M pages. This yields $\lceil B/M \rceil$ runs, each M pages long and internally sorted.
- **Phase 1 onward, merging.** Use $M-1$ buffer pages as inputs, one per run, and the remaining page as an output buffer. Merge $M-1$ runs into one longer run using the k-way merge from the refresher. Repeat over all runs. Each merge pass reduces the number of runs by a factor of $M-1$. Continue until a single sorted run remains.

Each pass reads and writes the whole file: about $2B$ page I/Os per pass.



4.2 Why the run length is M and the fan-in is $M-1$

Two details students ask about. In Phase 0 all M pages hold input, because the sorted output can be written back over the same buffer. In the merge phases, one page must be reserved for the output being assembled (a page is the unit of writing, so we fill a page of merged output before writing it), which leaves $M-1$ pages for inputs, one per run being merged. That is why the fan-in is $M-1$ and not M.

4.3 Counting the cost

Concept: External sort cost formula

With B pages of data and M buffer pages:

$$\begin{aligned} \text{initial runs} &= \lceil B/M \rceil \\ \text{merge fan-in} &= M - 1 \\ \text{passes} &= 1 + \lceil \log_{M-1} \lceil B/M \rceil \rceil \\ \text{total I/O} &\approx 2B \times \text{passes} \end{aligned}$$

The "1" is Phase 0. Each merge pass divides the number of runs by $M-1$, so the number of merge passes is the logarithm base $M-1$ of the initial run count.

Worked example: Tracing it small: $B = 8$, $M = 3$

Eight pages, three buffers. Say each page holds one number for simplicity: 7, 2, 9, 4, 1, 8, 3, 6.

1. **Phase 0.** Read 3 pages (7, 2, 9), sort in memory, write run (2, 7, 9). Read (4, 1, 8), write (1, 4, 8). Read the last 2 pages (3, 6), write (3, 6). Three runs: $\lceil 8/3 \rceil = 3$. I/O so far: 8 reads, 8 writes.
2. **Pass 1.** Fan-in is $M - 1 = 2$. Merge runs 1 and 2: (1, 2, 4, 7, 8, 9). Run 3 has no partner, so it passes through untouched (or is copied; implementations vary). Now 2 runs. I/O: 8 more reads and writes.
3. **Pass 2.** Merge the two remaining runs: (1, 2, 3, 4, 6, 7, 8, 9). One run. Done.

Passes $= 1 + \lceil \log_2 3 \rceil = 1 + 2 = 3$. I/O $\approx 2 \times 8 \times 3 = 48$. Now give it $M = 4$: Phase 0 makes 2 runs, fan-in 3, so one merge pass finishes it: 2 passes, 32 I/Os. One extra buffer page cut the I/O by a third.

Worked example: The lecture exercise: $B = 1000$, $M = 11$

1. Runs after Phase 0: $\lceil 1000/11 \rceil = 91$.
2. Fan-in per merge pass: $M - 1 = 10$.
3. Passes: $1 + \lceil \log_{10} 91 \rceil = 1 + 2 = 3$. (After pass 1, 10 runs; after pass 2, 1 run.)
4. Total I/O $\approx 2 \times 1000 \times 3 = 6000$ page I/Os.

Now double the memory to $M = 22$: 46 runs, fan-in 21, $\lceil \log_{21} 46 \rceil = 2$ still, so 3 passes. Give it $M = 100$: 10 runs, fan-in 99, one merge pass: 2 passes, 4000 I/Os. And with $M = 1000$ the file fits in memory: one pass, in-memory sort, 2000 I/Os (read once, write once). The lever is M.

Trade-offs: Why not just add memory?

More memory means fewer, wider merges and fewer passes, and beyond B pages the sort is entirely in memory. But memory is shared: `work_mem`

In the real world. In PostgreSQL, `work_mem` is M. A sort that fits shows `Sort Method: quicksort Memory: 71680kB` in the plan; one that does not shows `Sort Method: external merge Disk: 8192kB`. Class Example 3 flips between the two by changing only `work_mem`.

is allocated *per sort operation per query*, so a setting of 256 MB with fifty concurrent queries each doing two sorts is 25 GB. Production systems set it modestly and raise it per session for known heavy queries. The buffer pool (Chapter 6) competes for the same RAM.

Summary

External merge sort: make $\lceil B/M \rceil$ sorted runs of M pages, then merge $M-1$ at a time until one run remains. Passes $= 1 + \lceil \log_{M-1} \lceil B/M \rceil \rceil$; I/O $\approx 2B\times$ passes. Memory is the lever: more buffers, fewer passes; enough buffers, no disk at all.

5 Join Algorithms

5.1 The join problem

Combine rows from R and S where a condition holds, usually an equality $R.x = S.y$ (an **equi-join**). Joins are often the most expensive part of a query, because the naive approach compares every row of R with every row of S: a million times a million is 10^{12} comparisons. Three classic algorithms avoid that, each best in a different situation. Throughout, B_R and B_S are the page counts of R and S, $|R|$ is the number of *rows* of R, and M is the number of buffer pages.

To make the algorithms concrete, here is a tiny instance. R is customers (id, name), S is orders (cust_id, amount), joined on `id = cust_id`.

R = customers

id	name
1	Sara
2	Amir
3	Lina

S = orders

cust_id	amount
3	15
1	40
1	90
2	70
3	25

Expected result, five joined rows: (Sara, 40), (Sara, 90), (Amir, 70), (Lina, 15), (Lina, 25).

5.2 Nested-loop join

Mechanism: Nested-loop join, three variants

Simple: for each row r of R (the *outer*), scan all of S (the *inner*) and emit (r, s) for every match.

```
for r in R:                  # outer: read R once
    for s in S:              # inner: read S once PER ROW of R
        if r.x == s.y: emit(r, s)
```

Cost: $B_R + |R| \cdot B_S$. Reading S once per *row* of R is brutal.
Block nested-loop: read R in chunks of $M-2$ pages (one buffer for the current S page, one for output). For each chunk, scan S once and join every R row in the chunk against every S row. Cost: $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$. Now S is read once per *chunk* of R.
Index nested-loop: if S has an index on the join key, for each row of R look up its matches directly instead of scanning S. Cost: $B_R + |R| \cdot$ (index lookup cost), about $B_R + |R| \cdot (h + \text{matches})$.

Worked example: Block nested-loop on the tiny instance

Say $M = 4$, so chunks of $M - 2 = 2$ customers. Chunk 1 is (Sara, Amir): scan all five orders; order (3, 15) matches neither; (1, 40) and (1, 90) match Sara; (2, 70) matches Amir; (3, 25) matches neither. Chunk 2 is (Lina): scan all five orders again; (3, 15) and (3, 25) match. Orders was read twice, once per chunk.

Worked example: The lecture exercise: $B_R = 100$, $B_S = 1000$, $M = 12$

Chunks of R: $\lceil 100/10 \rceil = 10$. Cost $= 100 + 10 \times 1000 = 10{,}100$ page reads. Two ways to cut it: make the *smaller* table the outer (already done here; with S as outer it would be $1000 + 100 \times 100 = 11{,}000$), and give it more memory (with $M = 102$, one chunk: $100 + 1000 = 1100$). For the lab's paper exercise, $B_R = 200$, $B_S = 2000$, $M = 22$: $\lceil 200/20 \rceil = 10$ chunks, cost $200 + 10 \times 2000 = 20{,}200$.

When it wins: one input is small (a handful of pages fits in one chunk, so the inner is scanned once), or the inner has a selective index on the join key. It is also the only one of the three that handles arbitrary join conditions (inequalities, `LIKE`, anything), because it just tests the predicate.

Key insight. Always put the smaller input on the outside of a nested loop: the outer is read once, the inner is read once per outer chunk.

5.3 Sort-merge join

Mechanism: Sort-merge join

1. Sort R on the join key and sort S on the join key (external merge sort, Section 4).
2. Merge: walk both sorted inputs with one pointer each. If the keys differ, advance the pointer with the smaller key. If they are equal, emit every combination of the equal-key rows from both sides, then advance both.

Cost $\approx \text{sort}(R) + \text{sort}(S) + (B_R + B_S)$ for the final merge pass.

Worked example: Sort-merge on the tiny instance

Sorted R: (1, Sara), (2, Amir), (3, Lina). Sorted S: (1, 40), (1, 90), (2, 70), (3, 15), (3, 25).

R pointer	S pointer	Compare	Action
1 Sara	1, 40	equal	emit (Sara, 40)
1 Sara	1, 90	equal	emit (Sara, 90)
1 Sara	2, 70	$1 < 2$	advance R
2 Amir	2, 70	equal	emit (Amir, 70); advance both
3 Lina	3, 15	equal	emit (Lina, 15)
3 Lina	3, 25	equal	emit (Lina, 25); S exhausted

Each input was read exactly once during the merge, in order. (When *both* sides have several rows with the same key, the merge must back up on one side to produce all pairs; that is the one wrinkle in the implementation.)

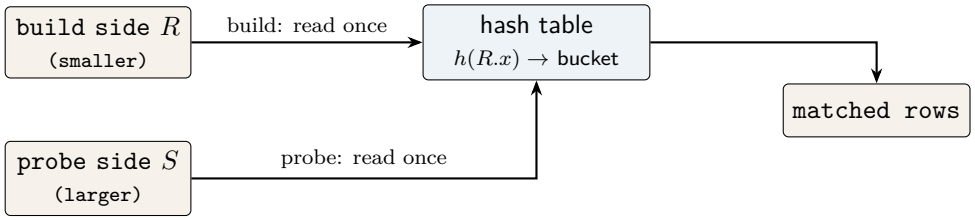
When it wins: the inputs are already sorted (from an earlier sort, or because an index scan delivers them in key order, as in Class Example 2 where customers arrive sorted via the primary-key index), or the query wants sorted output anyway (`ORDER BY` on the join key comes free). It also handles range and inequality joins such as `R.x < S.y`, which hashing cannot.

5.4 Hash join

Mechanism: Hash join

1. **Build.** Read the smaller input R once and insert every row into an in-memory hash table keyed on $h(R.x)$.
2. **Probe.** Read the larger input S once. For each row s, compute $h(s.y)$, look in that bucket, and emit (r, s) for every r there with $r.x = s.y$.

If the hash table fits in memory: cost $\approx B_R + B_S$, each input read exactly once. This is the minimum any join could possibly cost.



Worked example: Hash join on the tiny instance

Build on customers (the smaller side): hash table $\{1 \mapsto$ Sara, $2 \mapsto$ Amir, $3 \mapsto$ Lina$\}$. Probe with orders in file order: (3, 15) probes bucket 3, finds Lina, emits (Lina, 15). (1, 40) finds Sara. (1, 90) finds Sara. (2, 70) finds Amir. (3, 25) finds Lina. Five rows, each input read once. Notice the output comes out in *orders* order, not sorted by anything; hash join gives no ordering guarantee.

When the build side does not fit: Grace hash join. If R has more pages than the buffer can hold, partition first. Read R once, hashing each

row with a *partitioning* function h_1 into $M-1$ output buckets written to disk; do the same to S with the same h_1. Now any R row can only match S rows in the partition with the same h_1 value, so join partition i of R with partition i of S, one pair at a time, each small enough to build in memory. Cost: read and write both inputs to partition ($2(B_R + B_S)$), then read both once to join ($B_R + B_S$): about $3(B_R + B_S)$. In PostgreSQL plans this shows up as `Batches: 4` or more under the `Hash` node; `Batches: 1` means it fit.

When it wins: large equi-joins with no useful order and enough memory for the smaller side. It is usually the fastest option for big equality joins, which is why it is PostgreSQL's default choice in Class Example 2.

5.5 Which join wins?

Trade-offs: Choosing a join algorithm

Algorithm	Cost (pages)	Best when	Notes
Block nested-loop	$B_R + \lceil B_R/(M-2) \rceil B_S$	one input tiny	any join condition
Index nested-loop	$B_R + \lvert R \rvert \cdot (h +$ matches)	outer small, selective index on inner	the lab's "c.id < 10" case
Sort-merge	sort(R) + sort(S) $+ B_R + B_S$	inputs already sorted, or sorted output wanted	handles ranges
Hash	$B_R + B_S$ (fits) or $3(B_R + B_S)$ (Grace)	large equi-join, no useful order	usually fastest for big = joins

When each one is a disaster. Nested loop: both inputs large, nothing tiny and no index on the inner, so the inner is rescanned $\lceil B_R/(M-2) \rceil$ times; at 300 outer and 3,000 inner pages with $M = 32$ that is already 30,300 reads for a join a hash would do in 3,300. Sort-merge: neither input is sorted and no sorted output is wanted, so you pay two full external sorts, several passes over both tables, before the merge even starts. Hash: an inequality join (`<`, `BETWEEN`), which hashing cannot do at all; or a build side that badly overflows memory, so the join partitions both inputs to disk and reads everything three times (`Batches` > 1 in the plan). Each algorithm is the best choice in one of the other two's disasters, which is why a database keeps all three.

There is no single best join. The optimiser (Chapter 5) picks per query, using sizes, indexes, sortedness, and memory. Class Example 2 runs the *same* join of 500,000 rows three ways: hash 232 ms, merge 432 ms, nested loop 641 ms. Class Example 5 shows the choice flipping to nested loop when a `WHERE c.id < 10` shrinks the outer to nine rows: 0.26 ms.

Worked example: Pause and check: pick the join

1. Two large tables joined on equality, no indexes, no order needed.
2. Both inputs already sorted on the join key.
3. A small `customers` joined to a large `orders` that has an index on `cust_id`.
4. `WHERE r.start_date < s.event_date`.

1: Hash join, the best for a big equi-join with no useful order. 2: Sort-merge, skip straight to the merge. 3: Index nested-loop, for each customer look up its orders via the index. 4: Sort-merge or nested-loop; hashing cannot do inequalities.

Summary

Nested loop: outer read once, inner read once per outer chunk; put the small input outside; use an index on the inner when there is one. Sort-merge: sort both, then one linear merge; wins when order is already there or wanted. Hash: build on the smaller, probe with the larger, $B_R + B_S$ when it fits, $3\times$ that with Grace partitioning; wins for big equi-joins. No single best: sizes, indexes, order and memory decide.

6 Aggregation

`GROUP BY` faces the same structural question as duplicate elimination and joins: rows of the same group must be brought together. Two answers, the same two as always.

Trade-offs: Sort-based versus hash-based aggregation

Sort-based (GroupAggregate)	**Hash-based (HashAggregate)**
Sort the input by the group key.	Keep one hash-table entry per group.
Walk it: consecutive equal keys form a group; emit the aggregate when the key changes.	For each row, find its group's entry and update the running aggregate (count, sum, ...).
Streams after the sort; memory is one group at a time.	Emits only at the end; memory is one entry per group.
Good when the input is already sorted, or there are so many groups the hash table would not fit.	Good when groups are few, so the table is tiny.

Class Example 4 shows both on the same table: `GROUP BY country` (12 groups) runs as a `HashAggregate` with a 24 KB table in 6 ms; `GROUP BY name` (19,104 groups, nearly one per row) runs as a `Sort` then `GroupAggregate` in 26 ms. Same sort-versus-hash choice as joins, one level up. Note that a sort-based aggregate over sorted input is *not* a pipeline breaker: it emits each group as soon as the key changes. The hash-based one always is.

7 Seeing Operators in PostgreSQL

7.1 Reading a whole plan

Class Example 1 runs the Chapter 1 query on the lab database. Here is the real plan, lightly trimmed.

```
Sort  (actual time=121.84..122.07 rows=1996 loops=1)
  Sort Key: c.name
  Sort Method: quicksort  Memory: 205kB
  ->  HashAggregate  (actual time=120.93..121.31 rows=1996 loops=1)
        Group Key: c.name
        Batches: 1  Memory Usage: 721kB
        ->  Hash Join  (actual time=4.02..105.66 rows=50112 loops=1)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (actual time=0.01..37.95 rows=500000 loops=1)
              ->  Hash  (actual time=3.91..3.92 rows=2000 loops=1)
                    Buckets: 2048  Batches: 1  Memory Usage: 117kB
                    ->  Seq Scan on customers c  (actual time=0.01..3.52 rows=2000
    loops=1)
                          Filter: (country = 'MA'::bpchar)
                          Rows Removed by Filter: 18000
Execution Time: 122.38 ms
```

Technique: How to read it, bottom-up

1. **Seq Scan on customers with Filter.** Reads all 20,000 customers, keeps the 2,000 Moroccan ones (`Rows Removed by Filter: 18000`). Pipelined; the filter is applied per tuple.
2. **Hash.** Builds the hash table on those 2,000 rows: `Batches: 1` means it fit in memory (117 KB). *Pipeline breaker*: it must finish before the join can start.
3. **Seq Scan on orders.** All 500,000 rows, streamed.
4. **Hash Join.** Probes with each order; 50,112 joined rows come out (about one tenth, since one tenth of customers are Moroccan). Pipelined on the probe side.
5. **HashAggregate.** One entry per name, 1,996 groups, summing amounts. *Pipeline breaker.*
6. **Sort.** Orders the 1,996 result rows by name, in memory (`quicksort Memory: 205kB`). *Pipeline breaker*, but tiny.

The two-number `actual time=a..b` means: first row produced at a ms, last row at b ms. For the Hash node both are 3.9: it produces nothing until it is done. For the Hash Join, 4.02..105.66: first output right after the build finished, last output a hundred milliseconds later. That gap between first and last is what "streaming" looks like in numbers.

7.2 Forcing alternatives

```
SET enable_hashjoin  = off;    -- planner falls back to merge join
SET enable_mergejoin = off;    -- and then to nested loop
SET work_mem = '64kB';         -- force an on-disk external sort
RESET ALL;                     -- always, between experiments
```

These are teaching switches; in production the planner's choice is usually right. Class Example 2 shows the three plans for the full customers-orders join. The merge-join plan is worth reading:

```
Merge Join  (actual time=247.30..402.81 rows=500000 loops=1)
  Merge Cond: (c.id = o.cust_id)
  ->  Index Scan using customers_pkey on customers c  (rows=20000)
  ->  Sort  (actual time=247.21..331.58 rows=500000 loops=1)
        Sort Key: o.cust_id
        Sort Method: external merge  Disk: 8184kB
        ->  Seq Scan on orders o  (rows=500000)
Execution Time: 431.90 ms
```

Every idea of the chapter is in these seven lines. Customers arrive already sorted, for free, from the primary-key index scan. Orders must be sorted, and 500,000 rows do not fit in the default `work_mem`, so it is an *external merge* that spilled 8 MB to disk. The Sort's first row appears at 247 ms: that is the pipeline breaker, and the join cannot start until then. The merge itself then takes 155 ms. Compare the hash join at 232 ms total: it had no sort to wait for.

And the nested-loop plan:

```
Nested Loop  (actual time=0.03..612.44 rows=500000 loops=1)
  ->  Seq Scan on customers c  (rows=20000 loops=1)
  ->  Index Scan using orders_cust_id_idx on orders o  (rows=25 loops=20000)
        Index Cond: (cust_id = c.id)
```

`loops=20000`: the inner index scan ran once per customer, finding 25 orders each time. That is index nested-loop exactly as in Section 5, and 20,000 index descents is why it is slowest here. But cut the outer to nine customers (Class Example 5, `WHERE c.id < 10`) and the same plan finishes in 0.26 ms, a thousand times faster than the hash join would for that query. Same algorithm, different sizes, different winner.

8 Lab 4 Companion

Technique: Part by part

1. **Read a plan.** Use the walkthrough above. Operators bottom-up: Seq Scan (filter), Hash, Seq Scan, Hash Join, HashAggregate, Sort. Pipeline breakers: Hash, HashAggregate, Sort. The one that matters for cost is the Hash Join's build, though here it is tiny.
2. **Force the three joins.** Expect the ordering hash $<$ merge $<$ nested loop for the full join. Explain in terms of the table: hash reads each input once; merge pays an external sort of orders first; nested loop does one index descent per customer, 20,000 of them.
3. **External sort.** With `work_mem = '64kB'`, the Sort Method line reads `external merge Disk: 8192kB`; with `'256MB'`, `quicksort Memory: 71680kB`. Same algorithm choice as the $B = 8, M = 3$ trace versus fitting in memory.
4. **Hash versus sort aggregation.** `GROUP BY country`: HashAggre-

gate (12 groups). `GROUP BY name`: Sort then GroupAggregate (19,000 groups). Explain by the number of groups and hence the size of the hash table the planner would need.

5. **Size flips the join.** `c.id < 10`: Nested Loop with index scans, under a millisecond. Whole table: Hash Join, about 230 ms. Nothing changed but the row counts.
6. **Paper exercise.** $B_R = 200$, $B_S = 2000$, $M = 22$: $\lceil 200/20 \rceil = 10$ chunks; cost $200 + 10 \times 2000 = 20{,}200$ page reads. Show the chunk computation.

Debugging: If the plans do not behave

- **A forced setting seems ignored.** Some queries have no alternative plan (a nested loop with no index and no memory is still a nested loop). Try a larger or smaller query, and confirm you ran `RESET ALL;` between parts; a leftover `enable_hashjoin = off` silently distorts everything after it.
- **No external sort appears.** Lower `work_mem` further (`'64kB'` is the minimum) or sort a larger result set. A small table fits even in 64 KB.
- **Timings vary between runs.** The first run reads from disk, later runs from the buffer pool. Run twice and report the second; compare `Buffers` rather than milliseconds when you can.
- **Batches greater than 1 under Hash.** The build side did not fit in `work_mem`; that is Grace hash join partitioning to disk. Raise `work_mem` to see it drop to 1.

9 Tricks, Techniques and Classic Problems

Warning / Gotcha: Common student mistakes

- **Using $|R|$ where B_R belongs.** Simple nested loop is $B_R + |R| \cdot B_S$ (rows of R); block nested loop is $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$ (pages of R). Mixing them up changes the answer by a factor of rows-per-page.
- **Fan-in M instead of $M-1$.** One buffer page is the output; only $M-1$ runs merge at once.
- **Forgetting the Phase 0 pass.** Passes $= 1 + \lceil \log_{M-1} \text{runs} \rceil$; the 1 is run generation.
- **Counting reads only.** Each sort pass both reads and writes: $2B$ per pass.
- **Putting the large table on the outside of a nested loop.** The inner is re-read per outer chunk; the outer should be the small one.
- **Expecting a hash join to handle `<`.** Hashing finds equal keys only. Inequality joins need nested loop or sort-merge.
- **Reading `actual time=a..b` as a duration.** It is first-row time and last-row time; the operator's own work is roughly b minus its children's b.

- **Thinking every aggregate is a pipeline breaker.** HashAggregate is; GroupAggregate over sorted input streams group by group.

Worked example: Classic exam and interview problems

P1. External sort: $B = 1000$, $M = 11$. Runs, fan-in, passes, I/O?
91 runs; fan-in 10; $1 + \lceil \log_{10} 91 \rceil = 3$ passes; about $2 \times 1000 \times 3 = 6000$ I/Os.

P2. Smallest M that sorts $B = 1000$ pages in two passes?
Two passes means one merge pass, so all initial runs must merge at once: $\lceil 1000/M \rceil \leq M - 1$. $M = 33$ gives 31 runs and fan-in 32: yes. $M = 32$ gives 32 runs and fan-in 31: no. So $M = 33$. (Rule of thumb: about $\sqrt{B}$ buffers sort B pages in two passes.)

P3. Block nested-loop, $B_R = 100$, $B_S = 1000$, $M = 12$.
$\lceil 100/10 \rceil = 10$ chunks; $100 + 10 \times 1000 = 10{,}100$ reads. With S as outer: $1000 + \lceil 1000/10 \rceil \times 100 = 11{,}000$. Smaller outside is better.

P4. Hash join, $B_R = 100$, $B_S = 1000$, build fits in memory. Cost? And if it does not fit?
$100 + 1000 = 1100$. Grace: about $3 \times 1100 = 3300$. Still far below the nested loop's 10,100.

P5. Which join for each? (a) big equi-join, no indexes, no order; (b) both inputs sorted; (c) tiny outer, index on inner; (d) inequality condition.
(a) hash; (b) sort-merge; (c) index nested-loop; (d) sort-merge or nested loop, never hash.

P6. Name the pipeline breakers in: Sort → HashAggregate → Hash Join(Seq Scan, Hash(Seq Scan)).
Sort, HashAggregate, and the Hash build. The two Seq Scans and the join's probe side stream.

P7. Why does the database keep three join algorithms instead of only the fastest?
Because "fastest" depends on the situation. Sizes: hash shines on big equi-joins, nested loop on a tiny input. Indexes: an index on the join key makes index nested loop very cheap. Order: sorted inputs let sort-merge skip the expensive part. Memory: hash needs room for the build side. The optimiser chooses per query, which is the subject of Chapter 5.

P8. A plan shows `Sort Method: external merge Disk: 8184kB`. What happened and what would change it?
The sort input exceeded `work_mem`, so PostgreSQL ran external merge sort, writing runs to disk. Raising `work_mem` above the input size turns it into an in-memory quicksort; alternatively an index on the sort key lets an index scan deliver the rows already ordered, removing the sort entirely.

Summary: Chapter 4 in six lines

1. The executor runs the plan tree as iterators; tuples stream, except through pipeline breakers (sort, hash build, hash aggregate), which materialise and may spill to disk.
2. External merge sort: $\lceil B/M \rceil$ runs, merged $M - 1$ at a time; passes $= 1 +$

$\lceil \log_{M-1} \lceil B/M \rceil \rceil$; I/O $\approx 2B\times$ passes. Memory is the lever.

3. Nested loop: $B_R + \lceil B_R/(M-2) \rceil B_S$, small input outside, index on the inner if possible; handles any condition.
4. Sort-merge: sort both then one linear merge; wins when order is present or wanted; handles ranges.
5. Hash: build small, probe large, $B_R + B_S$ when it fits; the usual winner for big equi-joins.
6. Aggregation mirrors joins: sort to exploit order, hash to exploit equality. Next: how the optimiser chooses among all of these.